



Amazon DynamoDB

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Introduction to DynamoDB

What is NoSQL?

NoSQL is a type of database that is not relational and does not use SQL to query data for results

What is a Key/Value Store?

A form of data storage that has a **key** referencing a **value** and nothing more



What is a Document Store?

A form of data storage that has a **nested data** structure



Introduction to DynamoDB



DynamoDB is a NoSQL **key/value** and **document** database for internet-scale applications.

Features

- Fully managed
- Multi-region
- Multi-master
- Durable database
- Built-in security
- Backup and restore
- In-memory caching

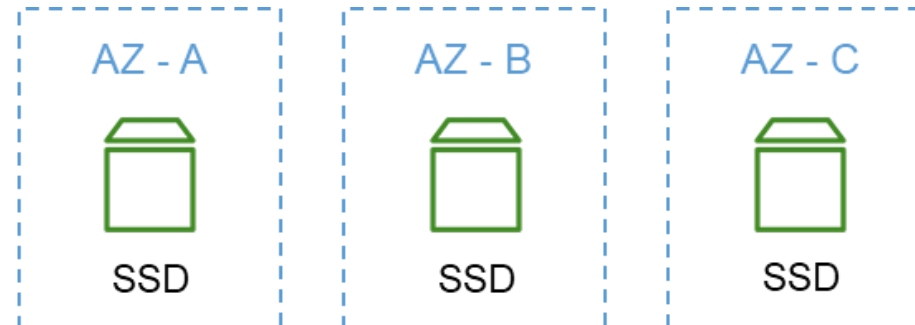
Provides

- Eventual Consistent Reads (default)
- Strongly Consistent Reads

Specify your read and write capacity per second. It just works at **whatever capacity you need** without tweaking anything.

Provisioned capacity

	Read capacity units	Write capacity units
Table	100	100
Estimated cost	\$58.04 / month (Capacity calculator)	



All data is stored on **SSD storage** and is spread across **3 different AZs**.

Anatomy of DynamoDB

Tables Tables contains rows and columns

Items The rows of data

Attributes The columns of data

Keys Identifying names of your data

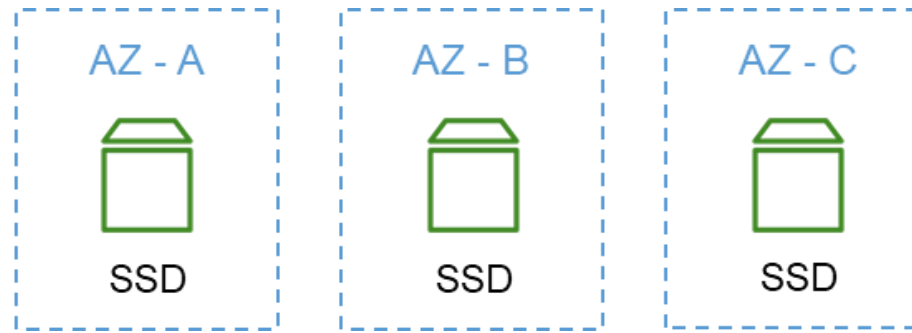
Values The actual data itself

The diagram shows a table with four columns: IMDB ID, Year, Title, and Box Office. The table is enclosed in a dashed blue border. Annotations include: a green arrow labeled 'Key' pointing to the 'IMDB ID' header; a red arrow labeled 'Item' pointing to the row containing 'Star Trek IV: The Voyage Home'; a yellow arrow labeled 'Value' pointing to the 'tt0102975' value in the 'IMDB ID' column; and a blue arrow labeled 'Table' pointing to the bottom-left corner of the table border.

IMDB ID	Year	Title	Box Office
tt0079945	1979	Star Trek: The Motion Picture	139000000
tt0084726	1982	Star Trek II: The Wrath of Khan	97000000
tt0088170	1984	Star Trek III: The Search for Spock	87000000
tt0092007	1986	Star Trek IV: The Voyage Home	133000000
tt0098382	1989	Star Trek V: The Final Frontier	63000000
tt0102975	1991	Star Trek VI: The Undiscovered Country	96.9
tt0111280	1994	Star Trek Generations	118000000
tt0117731	1996	Star Trek: First Contact	146000000
tt0120844	1998	Star Trek: Insurrection	117.8
tt0253754	2002	Star Trek: Nemesis	67.3

Table

DynamoDB - Read Consistency



When data needs to be updated, it has to write updates to all copies.
Data can be inconsistent when reading from a copy that has yet to be updated.

You can choose **the read consistency** in DynamoDB to meet your needs

Eventual Consistent Reads (DEFAULT)

- When copies are being updated, it is possible for you to read and be returned an inconsistent copy
- **Reads are fast**, but there is **no guarantee of consistent**
- All copies of data eventually **become generally consistent within a second**.

Strongly Consistent Reads

- When copies are being updated, and you attempt to read, it will not return a result until all copies are consistent.
- You have **a guarantee of consistency**, but the trade-off is **higher latency (slower reads)**.
- All copies of data **will be consistent within a second**

DynamoDB - Partitions

What is a Partition?

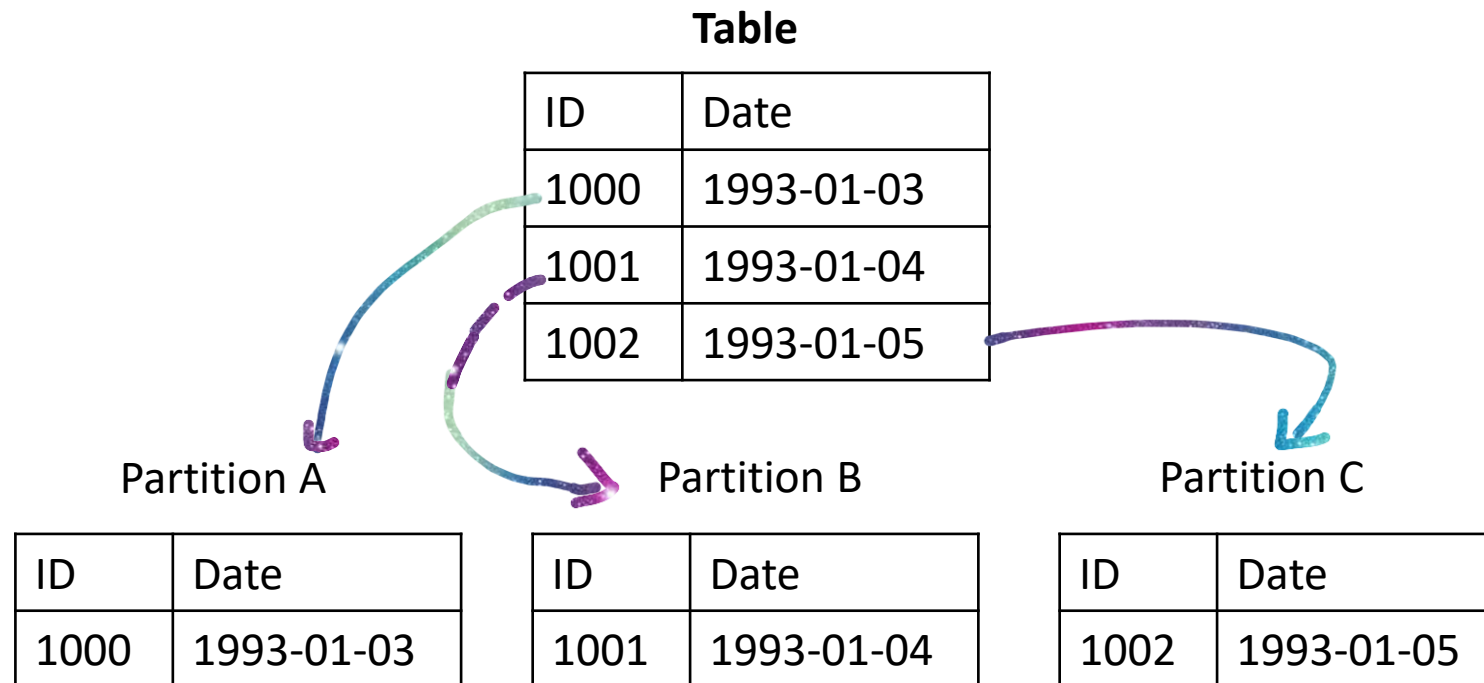
A *partition* is an allocation of storage for a table, backed by solid state drives (SSDs) and automatically replicated across multiple Availability Zones within an AWS Region.

AWS's definition



A *partition* is when **you slice your table up into smaller chunks of data** (a partition). **It speeds up reads** for very large tables by logically grouping similar data together.

My definition



DynamoDB – Partitions

DynamoDB **automatically** creates partitions for you as your data grows.

DynamoDB starts off with a single partition

There are 🙌 two cases where DynamoDB will create new partitions:

1. for every **10 GB** of data
2. when you exceed the RCUs or WCUs for a single partition

Each Partition has a maximum of **3000 RCUs** and **1000 WCUs**
Read capacity units Write capacity units

Provisioned capacity		
	Read capacity units	Write capacity units
Table	3000	1000
Estimated cost	<u>\$773.76 / month</u> (Capacity calculator)	

DynamoDB **evenly splits** the RCUs and WCUs across Partitions

DynamoDB – Primary Keys

When you create a table, you have to define a **Primary Key**

The primary key determines **where and how** your data will be stored in partitions.

⚠ The primary key **cannot** be changed later

Partition Key determines which partition data should be written to

Sort Key determines how data should be sorted on a partition.

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from hosts for scalability and availability.

ID String ▼
1 to 255 characters and case sensitive.

Sort key - *optional*
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search items with the same partition key.

Date String ▲
1 to 255 characters and case sensitive.

String
Binary
Number
String ✓

Table settings

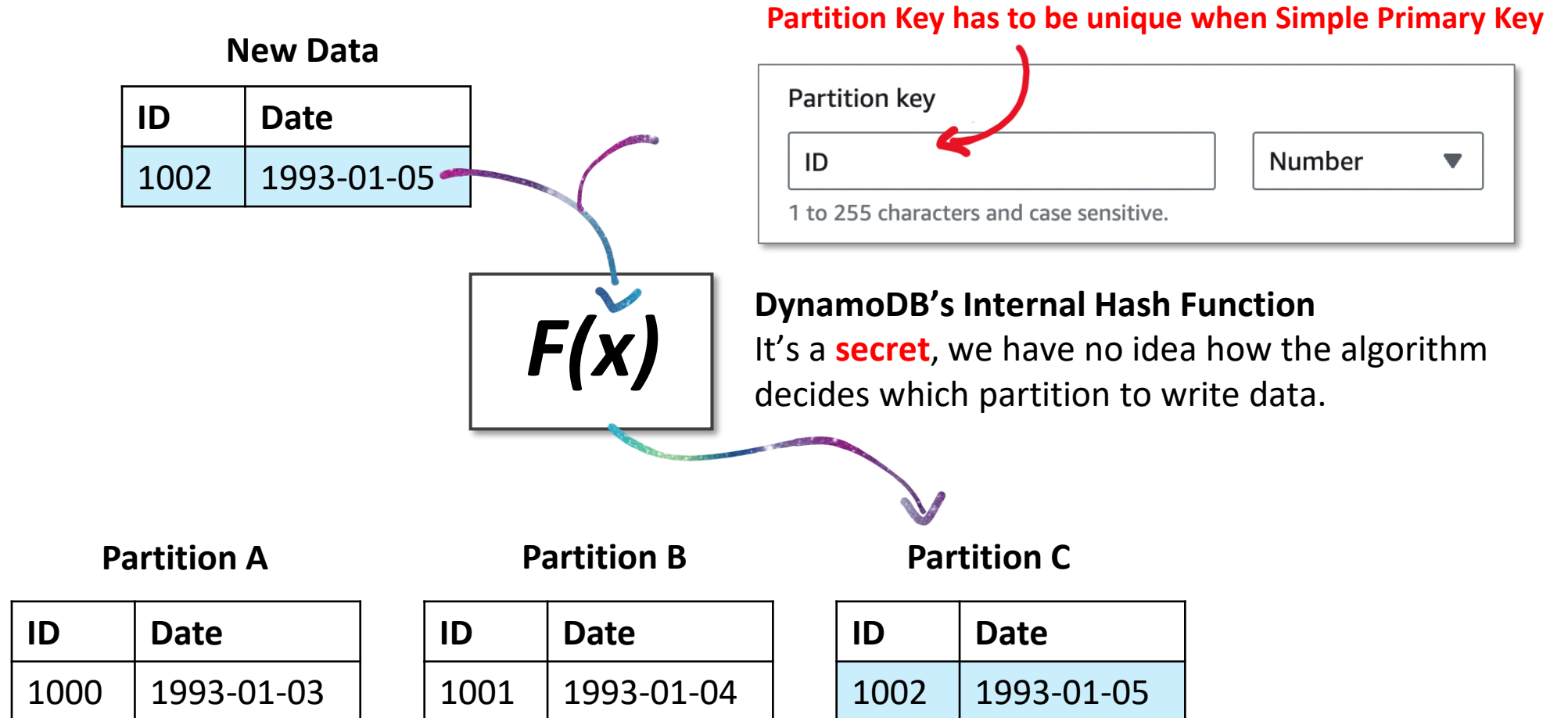
- Using only a Partition Key is called a **Simple** Primary Key
- Using both a Partition and Sort is called a **Composite** Primary Key

DynamoDB doesn't have a Date datatype so for dates you have to use a string.

DynamoDB – Simple Primary Key

A **Simple** Primary Key

How a Primary Key with **only a Partition Key** chooses which partition.



DynamoDB – Composite Primary Key

A **Composite** Primary Key

How a Primary Key with a **Partition** and **Sort Key** chooses which partition.

New Data

Alien	Name
Romulan	Tomalok

Partition key

Alien String ▼
1 to 255 characters and case sensitive.

Sort key - optional

Name String ▼
1 to 255 characters and case sensitive.

The combination of partition and sort key have to be unique.

$F(x)$

DynamoDB's Internal Hash Function

It's a **secret**, we have no idea how the algorithm decides which partition to write data.

Partition A

Alien	Name
Cardassian	Dukat
Cardassian	Dumar

Partition B

Alien	Name
Klingon	Worf
Klingon	Kor

Partition C

Alien	Name
Romulan	Tomalok
Romulan	Toreth
Romulan	Valdore

When using a sort key, the records that have the same Partition key will be kept together and sorted A-Z

DynamoDB – Primary Key Design

Simple Primary Keys

Only Partition Key

No two items can have the same Partition Key

✗	ID	Date
	1000	1993-01-03
✓	ID	Date
	1000	1993-01-03
	1001	1993-01-04

Composite Primary Keys

Partition + Sort Key

Two items can have the same partition key, but partition and sort key combined must be unique

✗	Alien	Name
	Romulan	Tomalok
✓	Alien	Name
	Romulan	Tomalok
	Romulan	Valdore

👉 Two things when designing your Primary Key

Distinct – The key should be as distinct (unique) as possible

Uniform – The key should evenly divide data

DynamoDB – Query and Scan

The best to learn what **Query** and **Scan** can do is to play around with the DynamoDB console

The screenshot shows the AWS DynamoDB console interface for a table named 'exampro-events'. The 'Scan or query items' section is active, with the 'Scan' radio button selected. A dropdown menu is open, displaying various comparison operators: 'Equal to' (selected with a checkmark), 'Not equal to', 'Less than or equal to', 'Less than', 'Greater than or equal to', 'Greater than', 'Between', 'Exists', 'Not exists', 'Contains', 'Not contains', and 'Begins with'. Below the dropdown, the 'Filters' section is visible, showing a table with columns 'Attribute name' and 'Type'. The 'Attribute name' column has a search input field with the placeholder 'Enter attribute...'. The 'Type' column has a dropdown menu with 'String' selected. There is an 'Add filter' button below the table. At the bottom of the console, there are 'Run' and 'Reset' buttons. On the right side of the console, there is an 'Autopreview' toggle switch and a 'View table' button.

You can choose
To **Query** or **Scan**

Explore the **sorting** options

DynamoDB – Query

- **Query** allow you to **find items** in a table based on primary key values
- You can query any table or secondary index that has a **composite** primary key (partition and sort key).
- By default, reads as **Eventually Consistent** (if you want Strongly Consistent set **ConsistentRead True**)
- By default, returns all attributes for items
- You can return specific attributes by using **ProjectExpression**
- By default, is sorted ascending (Use **ScanIndexForward False** to reverse or to descend)

```
aws dynamodb query \  
--table-name MusicCollection \  
--projection-expression "SongTitle" \  
--key-condition-expression "Artist = :v1" \  
--expression-attribute-values file://expression.json \  
--return-consumed-capacity TOTAL
```

```
# expression.json  
{  
  ":v1": {"S": "Wheatus"}  
}
```

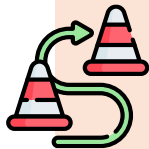


DynamoDB – Scan

- Scan **through all items** and then return one or more items through filters
- By default, returns all attributes for items
- Scans can be performed on tables and secondary indexes
- Can return specific attributes by using ProjectionExpression
- Scan operations are sequential. You can speed up a scan through parallel scans using Segments and Total Segments parameters

```
aws dynamodb scan \  
--table-name MusicCollection \  
--filter-expression "Artist = :a" \  
--projection-expression "#ST, #AT" \  
--expression-attribute-names file://names.json \  
--expression-attribute-values file://values.json
```

```
# names.json  
{  
  "#ST": "SongTitle",  
  "#AT": "AlbumTitle"  
}  
# values.json  
{  
  ":a": {"S": "Wheatatus"}  
}
```



Avoid Scans When Possible

- Scans are much less efficient than running a query
- As a table grows, scans take longer to complete
- A large table can use all your provisioned throughput in a single scan

DynamoDB – Provisioned Capacity

Provisioned Throughput Capacity

is the **maximum** amount of capacity your application is allowed to **read** or **write** per second from a table or index.

Throughput is measure in capacity units:

- **RCUs** Is Read Capacity Units
- **WCUs** is Write Capacity Units

You can set DynamoDB for Provisioned to scale capacity up and down based on utilization.

If you go beyond your capacity, you'll get the error **ProvisionedThroughputExceededException**

This is known as **Throttling**. Requests that are throttled will be dropped (data loss)

Read/write capacity mode

☒ Provisioned (free-tier eligible)
☐ On-demand

Last change to on-demand mode: No read/write capacity mode changes have been made.
Next available change to on-demand mode: You can update to on-demand mode at any time.

Provisioned capacity

	Read capacity units	Write capacity units
Table	5	5
Estimated cost	\$2.91 / month (Capacity calculator)	

Auto Scaling

☒ Read capacity ☐ Write capacity

Target utilization %

Minimum provisioned capacity units

Maximum provisioned capacity units

☒ Apply same settings to global secondary indexes

DynamoDB – On-Demand Capacity

On-Demand Capacity is pay-per-request.
So, you pay only for what you use.

On-demand mode is good for:

- New tables with unknown workloads.
- Unpredictable application traffic.
- The ease of paying for only what you use.

The throughput is limited by the **default**
upper limits for a table:

- 40,000 RCUs
- 40,000 WCUs

Since there **is no hard limit** imposed by the user,
On-Demand could become very expensive based
on the emerging scenario.



Read/write capacity mode

☐ Provisioned (free-tier eligible)

☒ On-demand

Last change to on-demand mode: No read/write capacity mode changes have been made.

Next available change to on-demand mode: You can update to on-demand mode at any time.

Provisioned capacity

Not applicable because read/write capacity mode is on-demand.

Auto Scaling

Not applicable because read/write capacity mode is on-demand.

[Save](#) [Cancel](#)

DynamoDB – Calculating Reads

Read Capacity Units (RCU)

A *read capacity unit* represents:

- one strongly consistent read per second,
- or two eventually consistent reads per second,
- for an item up to 4 KB in size.

Read capacity units	
Table	10

4 KB of data or less

= 10 strongly consistent reads
or 20 eventually consistent reads

How to calculate RCUs for **strong**

1. Round data up to nearest 4.
2. Divide data by 4
3. Times by number of reads

50 reads at 40KB per item. $\frac{(40/4)}{10} \times 50 = 500$ RCUs

Round up 6 to 8

10 reads at 6KB per item. $\frac{(8/4)}{2} \times 10 = 20$ RCUs

Round up 17 to 20

33 reads at 17KB per item. $\frac{(20/4)}{5} \times 33 = 165$ RCUs

How to calculate RCUs for **eventual**

1. Round data up to nearest 4.
2. Divide data by 4
3. Times by number of reads
4. Divide final number by 2
5. Round up to the nearest whole number

50 reads at 40KB per item. $\frac{(40/4)}{10} \times 50 / 2 = 250$ RCUs

Round up 9 to 12

11 reads at 9KB per item. $\frac{(12/4)}{3} \times 11 / 2 = 16.5$ RCUs
Round up 16.5 to 17

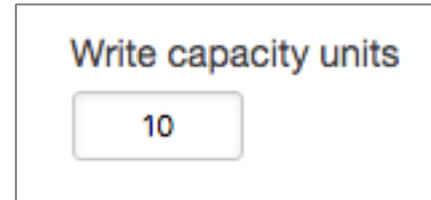
14 reads at 24KB per item. $\frac{(24/4)}{6} \times 14 / 2 = 42$ RCUs

DynamoDB – Calculating Writes

Write Capacity Units (WCU)

A *write capacity unit* represents:

- one write per second,
- for an item up to 1 KB



= 10 writes

1 KB of data or less

How to calculate WCUs

1. Round item size to the nearest 1 KB.
2. Times by number of writes

50 writes at 40KB per item. $40 \times 50 = 2000$ WCUs

11 writes at 1KB per item. $1 \times 11 = 11$ WCUs

Round up 500 B to 1KB

18 writes at 500 BYTES per item. $1 \times 18 = 18$ WCUs

DynamoDB – Global Tables

Amazon DynamoDB Global Tables provide a fully managed solution for deploying a **multi-region**, multi-master database without having to build and maintain your own replication solution.

```
aws dynamodb create-table \  
--table-name my-global-table \  
--attribute-definitions AttributeName=PK,AttributeType=S \  
--key-schema AttributeName=PK,KeyType=HASH \  
--provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1 \  
--stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES \  
--region ca-central-1
```

Create tables in each region

- Use the same name for every table
- Use KMS CMK
- Enable DynamoDB Streams
 - Set the Stream Type to be of New and Old Image

```
aws dynamodb create-global-table \  
--global-table-name my-global-table \  
--replication-group RegionName=us-east-1 RegionName=us-west-2 \  
--region ca-central-1
```

Enable global tables

DynamoDB – Transactions

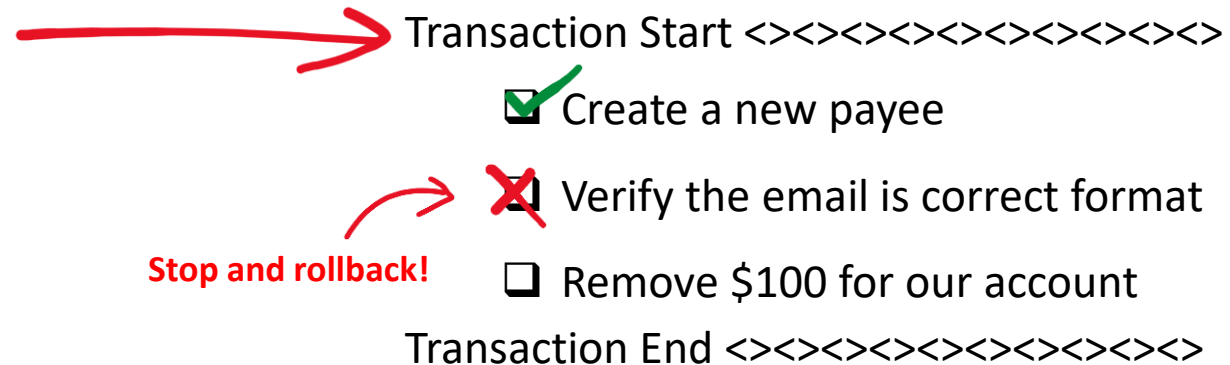
What is a transaction?

Represents a change that will occur to the database. If any dependent conditions fail, then a transaction will rollback as if the database changes never occurred.

Transactions are (ACID):

- Atomicity
- Consistency
- Isolation,
- Durability

If any of these steps fail
then the transaction
will immediately
Fail and roll back and
changes



DynamoDB – Transactions

DynamoDB offers the ability to perform transactions **at no additional cost** using the TransactWriteItems and TransactGetItems operations

DynamoDB transactions allow for **all-or-nothing** changes to multiple items both **within and across** tables

DynamoDB performs **two** underlying reads or writes of every item in the transaction:

1. one to prepare the transaction
2. one to commit the transaction.

These two underlying read/write operations are visible in your Amazon CloudWatch metrics.

You can use ConditionCheck with DynamoDB transactions to do preconditional check.

Check that an item exists or to check the condition of specific attributes of the item

DynamoDB – Transactions

TransactWriteItems: To write (put, update, delete) multiple items across multiple tables as an atomic transaction.

```
aws dynamodb transact-write-items \  
--transact-items '[{  
  "Put": {  
    "TableName": "YOUR_TABLE_NAME",  
    "Item": {  
      "PARTITION_KEY": {"S": "Item1"},  
      "Attribute1": {"S": "Value1"},  
      "Attribute2": {"N": "123"}  
    }  
  },  
  {  
    "Update": {  
      "TableName": "YOUR_TABLE_NAME",  
      "Key": {  
        "PARTITION_KEY": {"S": "Item2"},  
        "SORT_KEY": {"S": "SortValue"}  
      },  
      "UpdateExpression": "set Attribute3 = :val1",  
      "ExpressionAttributeValues": {  
        ":val1": {"S": "UpdatedValue"}  
      }  
    }  
  }  
}]'
```

DynamoDB – Transactions

TransactGetItems: To retrieve multiple items from one or more tables within a single transaction.

```
aws dynamodb transact-get-items \  
--transact-items '[  
  {"Get": {  
    "TableName": "YOUR_TABLE_NAME",  
    "Key": {  
      "PARTITION_KEY_NAME": {"S": "PartitionKeyValue1"},  
      "SORT_KEY_NAME": {"S": "SortKeyValue1"}  
    }}  
  },  
  {"Get": {  
    "TableName": "YOUR_TABLE_NAME",  
    "Key": {  
      "PARTITION_KEY_NAME": {"S": "PartitionKeyValue2"},  
      "SORT_KEY_NAME": {"S": "SortKeyValue2"}  
    }}  
  }  
]'
```

DynamoDB – Time To Live (TTL)

Time To Live (TTL) lets you have items in DynamoDB **expire** (deleted) at a given time.

TTL is great for keeping databases small and manageable and suited for temporary continuous data
eg. session data, event logs, usage patterns

Put in the **attribute** name, which will have a Number in DateTime format that will determine when this record will be deleted.

```
aws dynamodb update-time-to-live \  
--table-name my-table \  
--time-to-live-specification Enabled=true, AttributeName=expires_at
```



DynamoDB **doesn't have a DateTime datatype!**

To use TTL, you must use a Number that needs to be In **Epoch** format (Datetime represented as numbers)

Jan 05, 1993 10:00PM GMT



Convert to Epoch

726270962

A smaller database results in money saved!

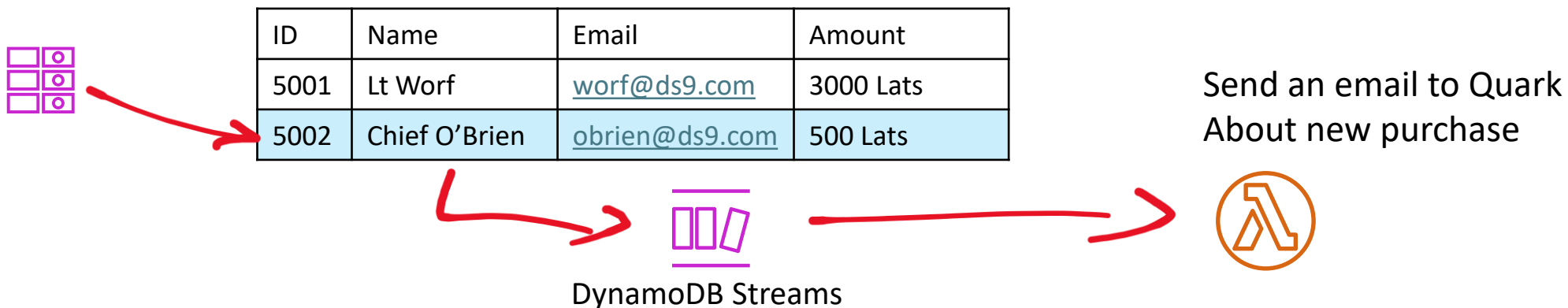
DynamoDB Streams

When you enable a **stream** on a table, DynamoDB captures every modification to data items so you can react to those changes

When an **Insert**, **Update**, or **Delete** occurs, the change will be captured and sent to  Lambda function

- Changes are sent in batches at time to your custom Lambda
- Changes are sent to your custom Lambda in near real-time
- Each stream record appears exactly once in the stream
- For each item that is modified, the stream records appear in the same sequence as the actual modifications

Ex. Every time a purchase is recorded, send an internal email.



DynamoDB Streams

```
aws dynamodb update-table \  
--table-name my-table \  
--stream-specification StreamEnabled=true,StreamViewType=NEW_IMAGE
```



Stream Type View determines the information that will be written to the stream for each item:

- **NEW_IMAGE** – The entire item, as it appears after it was modified, is written to the stream.
- **OLD_IMAGE** – The entire item, as it appeared before it was modified, is written to the stream.
- **NEW_AND_OLD_IMAGES** – Both the new and the old item images are written to the stream.
- **KEYS_ONLY** – Only the key attributes of the modified item are written to the stream.

DynamoDB – Errors

ThrottlingException

The rate of requests exceeds the allowed throughput. This exception might be returned if you perform any of the following operations too rapidly: **CreateTable**, **UpdateTable**, **DeleteTable**

ProvisionedThroughputExceededException

You exceeded your maximum allowed provisioned throughput for a table or for one or more global secondary indexes.

The AWS SDK will **automatically retry** with **Exponential Backoffs** when an error occurs: It will attempt the request again 50ms, 100ms, 200ms, up to a minute before stopping.



Remember these two Exceptions. They are common and likely to show up on the exam

DynamoDB – Indexes

What is an index?

A database index is a **copy** of selected columns of data in a database which is used to sort quickly

DynamoDB has two types of Indexes:

1. **LSI** - Local Secondary Index (can only be created with initial table)
2. **GSI** - Global Secondary Index

You should generally use Global over Local

Strong Consistency IS A Deciding Factor

A local secondary index **can provide** strong consistency

A global secondary index **cannot provide** strong consistency

DynamoDB – Local Secondary Indexes

What is a Local secondary index (LSI)?

A local secondary index is "local" in that every partition of a local secondary index is scoped to a base table partition **with the same partition key value**.

- The total size of indexed items for any one partition key value **can't exceed 10 GB**.
- **Its Shares provisioned throughput** settings for read and write activity with the table it is indexing.
- Limited to 5 per table (default)

DynamoDB – Local Secondary Indexes

LSIs are created with the initial table

LSIs cannot be added, modified, or deleted after creation

An LSI needs **both** a partition and a sort key

The partition key must **be the same** as the base table.

The sort key *should* **be different** from the base table

```
require 'aws-sdk-dynamodb' # v3
ddb = Aws::DynamoDB::Client.new
table_params = {
  table_name: 'YourTableName',
  attribute_definitions: [
    { attribute_name: 'PartitionKey', attribute_type: 'S' },
    { attribute_name: 'SortKey', attribute_type: 'N' },
    { attribute_name: 'LSI_SortKey', attribute_type: 'S' }
  ],
  key_schema: [
    { attribute_name: 'PartitionKey', key_type: 'HASH' },
    { attribute_name: 'SortKey', key_type: 'RANGE' }
  ],
  provisioned_throughput: {
    read_capacity_units: 1,
    write_capacity_units: 1
  },
  local_secondary_indexes: [
    {
      index_name: 'YourLSIName',
      key_schema: [
        { attribute_name: 'PartitionKey', key_type: 'HASH' },
        { attribute_name: 'LSI_SortKey', key_type: 'RANGE' }
      ],
      projection: {
        projection_type: 'INCLUDE',
        non_key_attributes: ['YourNonKeyAttribute']
      }
    }
  ]
}
ddb.create_table(table_params)
```

DynamoDB – Global Secondary Indexes

What is a Global secondary index (LSI)?

It's considered "global" because queries on the index can **span all of the data in the base table across all partitions.**

The indexes have no sizes restrictions (unlimited)

They have their own provisioned throughput settings. They consume capacity but not from the base table

Limited to 20 per table (default)

DynamoDB – Global Secondary Indexes

GSI can be added, modified, or deleted at any time.
The partition key should **be different** from the base table.
The sort key is optional but not required

```
require 'aws-sdk-dynamodb'

ddb = Aws::DynamoDB::Client.new(region: 'your-region')

update_params = {
  table_name: 'YourTableName',
  attribute_definitions: [
    { attribute_name: 'GSI_PartitionKey', attribute_type: 'S' }
  ],
  global_secondary_index_updates: [
    {
      create: {
        index_name: 'YourGSIName',
        key_schema: [
          { attribute_name: 'GSI_PartitionKey', key_type: 'HASH' }
        ],
        projection: { projection_type: 'ALL' },
        provisioned_throughput: {
          read_capacity_units: 1,
          write_capacity_units: 1
        }
      }
    }
  ]
}

ddb.update_table(update_params)
```

DynamoDB – LSI vs GSI

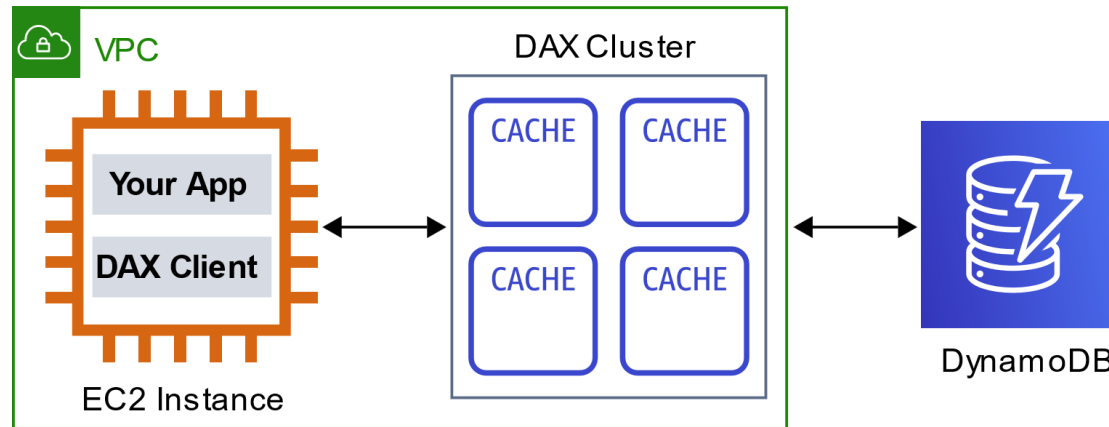
	Local Secondary Index	Global Secondary Index
Key Schema	Composite	★ Simple or Composite
Key Attributes	Partition key must be the same as the base table	★ Partition and Sort keys can be any attribute
Size Restriction Per Partition Key Value	all indexed items must be 10 GB or less.	★ Unlimited
Online Index Operations	create an index on table creation	★ Add, modify, or delete index at anytime
Queries and Partitions	query over a single partition, as specified by the partition key value in the query.	★ query over the entire table, across all partitions
Read Consistency	★ Strongly or eventual consistency	Only eventual consistency
Provisioned Throughput Consumption	Share capacity units with the base table	★ Has its own capacity
Projected Attributes	★ can request attributes that are not projected into the index	can only request the attributes that are projected into the index

DynamoDB – Accelerator (DAX)

DAX is a fully managed **in-memory cache** for DynamoDB that runs in a cluster.

DynamoDB response times can be in **single-digit milliseconds**.

DAX can reduce response times to **microseconds!!!**



- A DAX cluster consists of one or more nodes.
- Each node runs its own instance of the DAX caching software.
- One of the nodes serves as the primary node for the cluster.
- Additional nodes (if present) serve as read replicas.
- Your app can access DAX by specifying the endpoint for the DAX cluster.
- The DAX client software works with the cluster endpoint to perform intelligent load balancing and routing.
- Incoming requests are evenly distributed across all of the nodes in the cluster.

DynamoDB – DAX Use Cases

Dax is Ideal for:

Apps that require the **fastest possible response time** for reads eg. real-time bidding, social gaming, and trading applications

Apps that **read a small number of items** more frequently than others

Apps that are **read-intensive** but are also cost-sensitive.

Apps that require repeated reads against a large set of data.

Dax is *not* Ideal for:

Apps that require strongly consistent reads

Apps that do not require microsecond response times for reads or that do not need to offload repeated read activity from underlying tables

Apps that are **write-intensive** or that do not perform much read activity

Apps that are already using a different caching solution with DynamoDB and are using their own client-side logic for working with that caching solution.



**If you need a caching solution for the scenarios
Think of using ElastiCache**

DynamoDB – PartiQL

PartiQL is a **SQL-compatible query language**, to select, insert, update, and delete data in **Amazon DynamoDB**

```
require 'aws-sdk-dynamodb'
dynamodb_client = Aws::DynamoDB::Client.new

query = <<~PARTIQL
SELECT *
FROM "YourTableName"
WHERE "YourPartitionKey" = 'YourPartitionKeyValue'
PARTIQL

response = dynamodb_client.execute_statement({
  statement: query
})
puts response.items
```

DynamoDB – NoSQL WorkBench



NoSQL Workbench is cross-platform desktop application; it's a **visual development tool** that provides data modeling, data visualization, and query development features to help you design, create, query, and manage DynamoDB tables.

Data modeling

Design data models before you create your production tables

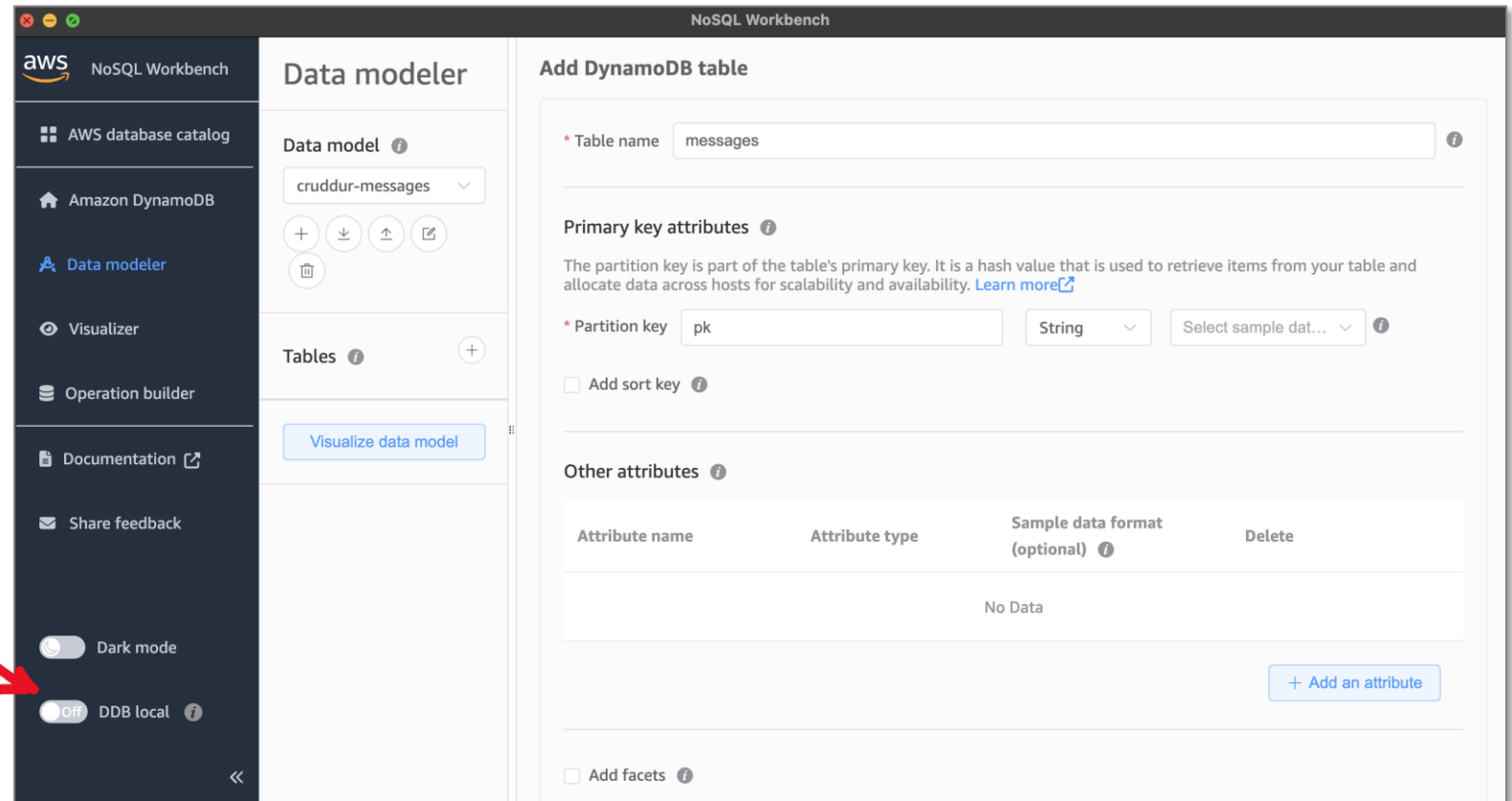
Data visualization

map queries and visualize the access patterns (facets) of the application without having to write code.

Operation building

view, explore, and query live datasets

Can be used along with DynamoDB Local so you can work against a non-production version of your data locally.



DynamoDB Local

DynamoDB Local allows you to run a local version of DynamoDB. This is useful for local development and reduce costs while developing DDB solutions

```
version: "3.8"
services:
  dynamodb-local:
    user: root
    command: "-jar DynamoDBLocal.jar -sharedDb -dbPath ./data"
    image: "amazon/dynamodb-local:latest"
    container_name: dynamodb-local
    ports:
      - "8000:8000"
    networks:
      - cruddur-net
    volumes:
      - "./docker/dynamodb:/home/dynamodblocal/data"
    working_dir: /home/dynamodblocal

volumes:
  db:
    driver: local
```

DynamoDB local is written in Java

The easiest way to run DynamoDB local is using docker utilizing the **docker-compose file**.

```
aws dynamodb list-tables --endpoint-url http://localhost:8000
```

To use DyanmoDB Local change your endpoint for API calls to local port you have mapped for DyanmoDB local

DynamoDB API Actions – Data Types

```
aws dynamodb put-item \  
--table-name TableName \  
--item '{  
  "PrimaryKey": {"S": "Value"},  
  "MyStrings": {"SS": ["Value1", "Value2", "Value3"]}  
}'
```



You have to indicate the Data Type in API calls

- **S** – String
- N – Number
- B – Binary
- BOOL – Boolean
- NULL – Null
- M – Map
- L – List
- **SS** – String Set
- NS – Number Set
- BS – Binary Set

DynamoDB API Actions – GetItem

GetItem operation returns a set of attributes for the item **with the given primary key**

```
aws dynamodb get-item \  
--table-name TableName \  
--key '{"PrimaryKey": {"S": "Value"}}'
```



If there is no matching item, GetItem does not return any data and there will be no Item element in the response.

GetItem provides an eventually consistent read by default.

If your application requires a strongly consistent read, set ConsistentRead to true.

DynamoDB API Actions – DeleteItem

Deletes a single item in a table **by primary key**.

```
aws dynamodb delete-item \  
--table-name TableName \  
--key '{"PrimaryKey": {"S": "Value"}}'
```



You can perform a **conditional delete operation** that deletes the item if it exists, or if it has an expected attribute value

```
aws dynamodb delete-item \  
--table-name TableName \  
--key '{"PrimaryKey": {"S": "Value"}}' \  
--condition-expression "attribute_exists(PrimaryKey)"
```



DynamoDB API Actions – UpdateItem

UpdateItem edits an existing item's attributes, or adds a new item to the table if it does not already exist.

```
aws dynamodb update-item \  
--table-name TableName \  
--key '{"PrimaryKey": {"S": "Value"}}' \  
--update-expression "SET Attribute1 = :val1" \  
--expression-attribute-values '{":val1":{"S":"NewValue"}}'
```

An **Update Expression** specifies **how UpdateItem will modify the attributes of an item**

DynamoDB API Actions – UpdateItem

Update Expression have the follow syntax format:

```
update-expression ::=  
  [ SET action [, action] ... ]  
  [ REMOVE action [, action] ... ]  
  [ ADD action [, action] ... ]  
  [ DELETE action [, action] ... ]
```

- SET — modifying or adding item attributes
- REMOVE — deleting attributes from an item
- ADD — updating numbers and sets
- DELETE — removing elements from a set

```
aws dynamodb update-item \  
--table-name Users \  
--key '{"UserId": {"S": "123"}}' \  
--return-values ALL_NEW \  
--expression-attribute-names '{  
  "#address": "Address",  
  "#email": "Email",  
  "#loginCount": "LoginCount",  
  "#skills": "Skills",  
  "#age": "Age"  
}' \  
--expression-attribute-values '{  
  ":newAddress": {"S": "123 Main St, Anytown, Canada"},  
  ":newEmail": {"S": "newemail@example.com"},  
  ":increment": {"N": "1"},  
  ":newSkill": {"L": [{"S": "AWS"}]}  
}' \  
--update-expression "  
SET  
#address = :newAddress,  
#email = :newEmail,  
#loginCount = #loginCount + :increment,  
#skills = list_append(#skills, :newSkill)  
REMOVE #age"
```

DynamoDB API Actions – PutItem

PutItem creates a new item, or replaces an old item with a new item

You can perform a conditional put operation

```
aws dynamodb put-item \  
--table-name TableName \  
--item '{  
  "PrimaryKey": {"S": "Value"},  
  "Attribute1": {"S": "ValueA"}  
}'
```

```
aws dynamodb put-item \  
--table-name TableName \  
--condition-expression "attribute_not_exists(PrimaryKey)"  
--item '{  
  "PrimaryKey": {"S": "Value"},  
  "Attribute1": {"S": "ValueA"}}  
,
```

- Empty String and Binary attribute values are allowed.
- Attribute values of type String and Binary must have a length greater than zero if the attribute is used as a key attribute for a table or index.

DynamoDB API Actions – BatchGetItem

```
aws dynamodb batch-get-item --request-items '{
  "RequestItems": {
    "TableName": {
      "Keys": [
        { "PrimaryKey": { "S": "Value1" } },
        { "PrimaryKey": { "S": "Value2" } }
      ]
    }
  }
}'
```

BatchGetItem is a single operation can retrieve up to 16 MB of data, which can contain as many as 100 items.

BatchGetItem returns a partial result if:

- the response size limit is exceeded,
- the table's provisioned throughput is exceeded,
- more than 1MB per partition is requested,
- or an internal processing failure occurs.

Remember that DynamoDB does not return items in any particular order.

By default, BatchGetItem performs eventually consistent reads on every table in the request. If you want strongly consistent reads instead, you can set ConsistentRead to true for any or all tables.

DynamoDB API Actions – BatchWriteItem

```
aws dynamodb batch-write-item --request-items '
{
  "RequestItems": {
    "TableName": [
      {
        "PutRequest": {
          "Item": {
            "PrimaryKey": { "S": "Value1" },
            "Attribute1": { "S": "ValueA" }
          }
        },
        {
          "DeleteRequest": {
            "Key": {
              "PrimaryKey": { "S": "Value2" }
            }
          }
        }
      ]
    }
  }
}
```

BatchWriteItem operation puts or deletes multiple items in one or more tables.

A single call to BatchWriteItem can transmit up to 16MB of data over the network, consisting of up to 25 item put or delete operations.

BatchWriteItem cannot update items.

If you perform a BatchWriteItem operation on an existing item, that item's values will be overwritten by the operation and it will appear like it was updated.

To update items, use the UpdateItem action.

DynamoDB API Actions – Count

You can count items during a scan or query by using **–select “COUNT”**

```
aws dynamodb scan \  
--table-name Users \  
--filter-expression "Status = :status" \  
--expression-attribute-values '":{"status": {"S": "Active"}}' \  
--select "COUNT"
```

```
{  
  "Count": 25,  
  "ScannedCount": 100  
}
```

The query will return two values:

- **Count:**
 - The number of items that match the criteria.
- **ScannedCount:**
 - The total number of items scanned before applying the filter

ExpressionAttributeValues

ExpressionAttributeValues is a map that contains the actual values that will be used in an expression so you can write more readable and maintainable code, as it separates the expression logic from the actual values.

These values are used to substitute placeholders in expressions eg.

- FilterExpression
- KeyConditionExpression
- ProjectionExpression
- UpdateExpression
- ConditionExpression

```
aws dynamodb scan \  
  --table-name Users \  
  --filter-expression "Age > :age" \  
  --expression-attribute-values '{":age":{"N":"25"}}'
```

```
aws dynamodb query \  
  --table-name Users \  
  --key-condition-expression "UserID = :userId" \  
  --expression-attribute-values '{":userId":{"S":"123"}}'
```

```
aws dynamodb update-item \  
  --table-name Users \  
  --key '{"UserID": {"S": "123"}}' \  
  --update-expression "SET Age = :age" \  
  --expression-attribute-values '{":age":{"N":"30"}}'
```

ConditionExpression

ConditionExpression lets you conditionally perform an action if the condition-expression is met.

```
aws dynamodb delete-item \  
--table-name ProductCatalog \  
--key '{"Id":{"N":"456"}}' \  
--condition-expression "(ProductCategory IN (:cat1, :cat2)) and (Price between :lo and :hi)" \  
--expression-attribute-values file://values.json
```

Built-in function you can use in a ConditionExpression

- attribute_exists (path)
- **attribute_not_exists** (path)
- attribute_type (path, type)
- size (path)
- begins_with (path, substr)
- contains (path, operand)
- if_not_exists (path, operand)



```
aws dynamodb put-item \  
--table-name ProductCatalog \  
--item file://item.json \  
--condition-expression "attribute_not_exists(Id)"
```

FilterExpression vs ProjectionExpression

FilterExpression

Apply additional filtering criteria to the items returned

It works as a **post-fetch filter**, meaning it processes the items after they have been read from the table but before they are returned to the client.

Eg. Filtering items where the Age attribute is greater than 25.

ProjectionExpression

Specify the attributes you want to retrieve from the table.

It reduces the amount of data retrieved by only fetching the specified attributes, which can **reduce cost and improve performance**.

Eg. Retrieving only the Name and Age attributes of items.

```
aws dynamodb query \  
  --table-name Users \  
  --key-condition-expression "UserID = :userId" \  
  --filter-expression "Status = :status" \  
  --projection-expression "Name, Age" \  
  --expression-attribute-values '{":userId":{"S":"123"}, ":status":{"S":"Active"}}'
```